

Task 7 — String Processing for Data Cleaning

1. Description

Practice string operations such as trimming spaces, changing case, replacing characters, splitting text, and extracting information. These operations are commonly used when text fields contain inconsistent formatting or unwanted characters.

2. Objective

Build basic text-cleaning skills needed for real-world datasets. The focus is on using Python string methods to make text values consistent and easier to analyze.

3. Tools

- Python
- Jupyter Notebook

4. Deliverables

- A notebook demonstrating common string methods
- A small text-cleaning function
- Examples of cleaning names, emails, and categories

5. Understanding String Cleaning

Raw text data often contains leading or trailing spaces, inconsistent capitalization, unwanted characters, or different formats for the same category. String cleaning applies simple transformations so equivalent values follow a consistent format.

6. Common String Methods

strip() — removes leading and trailing spaces.

lower() — converts letters to lowercase.

upper() — converts letters to uppercase.

replace() — replaces specified text or characters.

split() — separates a string into a list of parts.

```
text = "  Abhishek Kumar  "

print("Original:", repr(text))
print("strip():", text.strip())
print("lower():", text.lower())
print("upper():", text.upper())
print("replace():", text.replace("Abhishek", "A."))
print("split():", text.strip().split())
```

7. Why strip() Is Important

Unwanted spaces can make otherwise identical values appear different. For example, " Rahul " and "Rahul" should normally represent the same name. `strip()` removes the extra spaces at the beginning and end.

```
name = "  Rahul Sharma  "
clean_name = name.strip()
print(clean_name)
```

8. Changing Case for Standardization

Case conversion is useful when values may appear as ELECTRONICS, Electronics, or electronics. `lower()` or `upper()` can make the format consistent.

```
category = "ELECTRONICS"
print(category.lower())

department = "information technology"
print(department.upper())
```

9. Replacing Characters or Text

`replace()` is useful for removing or changing unwanted characters. For example, hyphens in a phone number can be removed when a numeric-only representation is required.

```
phone = " +91-98765-43210 "
clean_phone = phone.strip().replace("-", "")
print(clean_phone)
```

10. Removing Extra Internal Spaces

`strip()` removes spaces at the ends, while `split()` followed by `join()` can also normalize multiple spaces inside text.

```
raw = " Data Science "
clean = " ".join(raw.strip().split())
print(clean)
```

11. Splitting Text

`split()` divides a string into separate parts. It is useful for names, words, identifiers, and other structured text.

```
full_name = "Abhishek Kumar"
parts = full_name.split()

print("First name:", parts[0])
print("Last name:", parts[1])
```

12. Extracting Information from an Email

An email address contains a username and domain separated by `@`. `split()` can separate these parts for basic text processing.

```
email = "abhishek@example.com"
username, domain = email.split("@")

print("Username:", username)
print("Domain:", domain)
```

13. Small Text-Cleaning Function

A function makes the cleaning process reusable. The same function can be applied to multiple records instead of repeating the cleaning logic.

```
def clean_text_records(records):
    cleaned = []

    for row in records:
        cleaned.append({
            "name": row["name"].strip().title(),
            "email": row["email"].strip().lower(),
            "category": row["category"].strip().lower()
        })

    return cleaned
```

14. Cleaning Names, Emails and Categories

Names: trim spaces and use title case for readable formatting.

Emails: trim spaces and use lowercase for consistent text representation.

Categories: trim spaces and standardize case so equivalent categories are easier to group.

15. Customer Dataset Example

```
customers = [
    {"name": "  Rahul Sharma ",
     "email": "  RAHUL@EXAMPLE.COM ",
     "category": "  electronics "},

    {"name": "Priya Singh",
     "email": " priya@example.com",
     "category": "ELECTRONICS"}
]

cleaned = clean_text_records(customers)

for row in cleaned:
    print(row)
```

16. Expected Cleaning Logic

Field	Raw Example	Clean Format
Name	Rahul Sharma	Rahul Sharma
Email	RAHUL@EXAMPLE.COM	rahul@example.com
Category	ELECTRONICS	electronics

17. Employee Dataset Example

Employee data can also contain inconsistent capitalization and spaces. The same string cleaning principles can standardize names and department values.

```
employees = [
    {"name": " mohan kumar ", "department": " IT "},
    {"name": "SNEHA GUPTA", "department": " human resources "}
]

clean_employees = [
    {
        "name": e["name"].strip().title(),
        "department": e["department"].strip().title()
    }
    for e in employees
]

print(clean_employees)
```

18. Standardizing Categories

The goal is to make different text representations consistent. For example, Electronics, ELECTRONICS, and electronics can be converted to the same lowercase representation.

```
categories = [
    " Electronics ",
    "ELECTRONICS",
    "electronics",
    " Home Appliances "
]

standardized = [c.strip().lower() for c in categories]
print(standardized)
```

19. Complete Data-Cleaning Workflow

- Step 1:** Inspect the raw text.
- Step 2:** Remove unwanted leading/trailing spaces.
- Step 3:** Standardize capitalization.

- Step 4:** Replace unwanted characters where required.
- Step 5:** Split strings when information must be extracted.
- Step 6:** Standardize categories.
- Step 7:** Check the cleaned output for consistency.

20. Key Learning Points

- String methods can solve common text-quality problems.
- Consistent formatting makes text easier to analyze.
- Functions make repeated cleaning tasks reusable.
- Names, emails, categories, and departments can be standardized using simple Python operations.

21. Interview Questions & Answers

Q1. Why is string cleaning important in data science?

String cleaning improves consistency and data quality before analysis.

Q2. What does `strip()` do?

It removes leading and trailing whitespace from a string.

Q3. How can you convert a string to lowercase?

Use the `lower()` method, for example: `text.lower()`.

Q4. What is the purpose of `replace()`?

It replaces a specified substring or character with another value.

Q5. What does `split()` do?

It divides a string into parts and returns the parts as a list.

Q6. Why standardize categorical values?

It helps equivalent values use the same representation and reduces inconsistent categories.

Q7. Why use a cleaning function?

A function makes the same cleaning logic reusable across multiple records or datasets.

22. Top 5 Skills

1. Python Programming
2. String Manipulation
3. Data Cleaning
4. Data Standardization
5. Problem Solving & Data Preparation

23. Assessment Checklist

Requirement	Status
Common string methods	Completed
Text-cleaning function	Completed
Cleaning names	Completed
Cleaning emails	Completed
Standardizing categories	Completed

Customer dataset example	Completed
Employee dataset example	Completed
Interview questions	Completed